

Phase 5: Project Development Phase

Code-Layout, Readability and Reusability

1. Architectural Code Layout

The ComicCraft codebase is structured strictly according to the Single Responsibility Principle (SRP) and separation of concerns. The directory layout ensures that configuration, backend routing, AI service adapters, static assets, and presentation templates remain cleanly decoupled:

```
magesh/ (Project Root)
├── app/                # Core Application Package
│   ├── __init__.py    # Package initializer
│   ├── main.py        # Server entry point & static mount
│   ├── routes.py      # API endpoints & controller logic
│   ├── gemini_flash.py # Gemini 1.5 Flash outline generator
│   ├── gemini_pro.py  # Gemini 1.5 Pro narrative & dialogue
│   ├── image_generator.py # Visual inking & Diffusion pipeline
│   ├── layout_builder.py # Panel layout binder & metadata extractor
│   └── exporters.py    # FPDF2 multi-page PDF compilation engine
├── templates/         # Jinja2 Dynamic HTML Templates
│   ├── index.html     # Creation form & story preset picker
│   ├── comic_preview.html # 5-Panel sequential story preview
│   └── export_success.html # Download confirmation & celebratory UI
├── static/            # Static Assets
│   ├── css/
│   │   └── style.css  # Custom modern comic design system
│   ├── js/
│   │   └── main.js    # Form presets & dynamic loading states
│   ├── panels/        # Auto-generated panel illustrations (.png)
│   ├── exports/       # Auto-generated comic PDFs (.pdf)
│   ├── fonts/         # DejaVuSans.ttf & DejaVuSans-Bold.ttf
│   └── images/        # Background references & brand media
├── .env               # Environment secret configuration
├── .env.example       # Environment configuration template
├── requirements.txt   # Managed project dependencies
└── README.md          # Comprehensive project guide
```

2. Code Readability & Style Adherence

1. PEP 8 Compliance: All Python modules adhere to standard PEP 8 naming conventions, function length standards, and indentation.
2. Explicit Type Hints: All core function signatures utilize Python typing constructs (List, Dict, Optional, Any), promoting code clarity and static analysis.
3. Descriptive Docstrings: Every module, class, and public function includes Google/NumPy style docstrings explaining its purpose, arguments, and return types.
4. Structured Error Handling: API endpoints wrap operations in explicit try/except blocks, returning descriptive HTTP status codes and error messages instead of generic server faults.

3. Reusability & Extensibility Patterns

1. Modular AI Service Connectors:

- gemini_flash.py and gemini_pro.py are standalone functions that can be imported into CLI tools, batch processing scripts, or alternative web frameworks.

2. Deterministic Image Generation & Caching:

- `generate_image(prompt)` sanitizes and hashes prompts, returning existing cached files when available to conserve compute and avoid redundant generation.

3. Pluggable PDF Exporter:

- `save_pdf(layout)` consumes an arbitrary list of panel dictionaries, allowing the same exporter to support 3-panel, 5-panel, or 10-panel comic strips without code modification.

4. Decoupled API vs. Web Interface:

- `routes.py` provides both server-rendered Jinja2 responses for browsers (`/generate`) and pure JSON responses for third-party mobile or web clients (`/generate-comic/json`).